

NAME : KRISH GUPTA

ROLL NO : 60 D15C

EXPERIMENT 5 : Implement Support Vector Machine (SVM) for Classification with Hyperparameter Tuning

1. Dataset Source

Dataset Name: Resume Dataset

Source: Kaggle

Link: <https://www.kaggle.com/datasets/snehaanbhawal/resume-dataset/>

This dataset contains resume text data categorized into different job domains and is used for multi-class text classification tasks.

2. Dataset Description

Overview

The Resume Dataset consists of resume text samples labeled according to job categories such as Data Science, HR, Software Developer, Mechanical Engineer, etc.

The objective is to classify resumes into the correct job category using machine learning.

Dataset Size

- Total Instances: ~960 resumes
 - Input Feature: Resume (Text data)
 - Target Variable: Category
 - Type: Multi-class classification
-

Target Variable

The Category column contains job categories such as:

- Data Science
- HR

- Mechanical Engineer
- Civil Engineer
- Business Analyst
- Software Developer
- Operations Manager
- Sales
- Advocate
- Testing
- Web Designing
- Blockchain
- Electrical Engineering
- DevOps Engineer

Each resume belongs to exactly one category.

Dataset Characteristics

- Unstructured text data
- Requires text preprocessing
- Multi-class classification problem
- Balanced distribution across categories

Since the dataset is text-based, feature extraction techniques such as **TF-IDF vectorization** are required before applying SVM.

3. Mathematical Formulation of Support Vector Machine (SVM)

Support Vector Machine is a supervised learning algorithm used for classification and regression.

The objective of SVM is to find the **optimal hyperplane** that maximizes the margin between different classes.

Linear SVM Equation

The decision boundary is defined as:

$$w \cdot x + b = 0$$

Where:

- w = weight vector
- x = feature vector
- b = bias

Optimization Objective SVM

minimizes:

$$\frac{1}{2} \|w\|^2$$

Subject to:

$$y_i(w \cdot x_i + b) \geq 1$$

Where:

- y_i = class label
- x_i = feature vector

Soft Margin SVM

For non-linearly separable data:

$$\min \frac{1}{2} \|w\|^2 + C \sum \xi_i$$

Where:

- C = Regularization parameter
- ξ_i = Slack variables

Kernel Trick

If data is not linearly separable, SVM uses kernel functions:

- Linear Kernel

- Polynomial Kernel
 - RBF (Radial Basis Function) Kernel
-

4. Algorithm Limitations

1. Computationally Expensive

SVM training becomes slow for very large datasets.

Condition:

When number of samples is very high.

2. Choice of Kernel

Selecting wrong kernel reduces performance.

Condition:

If linear kernel is used for non-linear text patterns.

3. Sensitive to Hyperparameters

Improper C or gamma can cause underfitting or overfitting.

4. Less Interpretable

Unlike Decision Trees, SVM does not provide clear decision rules.

5. Methodology / Workflow

Step 1: Data Loading

Load dataset (Resume.csv) into Pandas DataFrame.

Step 2: Text Preprocessing

- Convert text to lowercase
- Remove punctuation

- Remove stopwords
 - Apply tokenization
-

Step 3: Feature Extraction

Use **TF-IDF Vectorizer** to convert resume text into numerical vectors.

$$TF - IDF = TF \times IDF$$

Where:

- TF = Term Frequency
 - IDF = Inverse Document Frequency
-

Step 4: Train-Test Split

dataset into:

- 80% Training
 - 20% Testing
-

Step 5: Model Training

Train Support Vector Machine classifier using:

- Linear kernel
 - RBF kernel
-

Step 6: Hyperparameter Tuning

Use GridSearchCV to tune:

C (Regularization parameter)

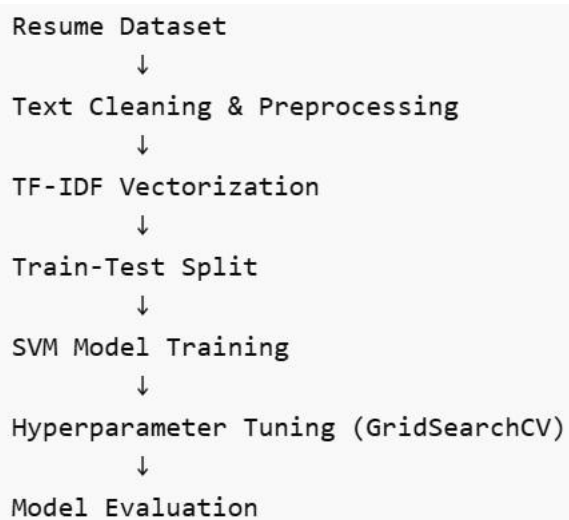
- kernel
- gamma (for RBF)

Step 7: Evaluation Evaluate

using:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

Workflow Diagram



6. Performance Analysis

Evaluation Metrics Used

- Accuracy
- Macro Precision
- Macro Recall
- Macro F1-Score

Typical Results

Metric	Value
--------	-------

Accuracy	~97%
----------	------

Precision	~96%
-----------	------

Recall	~96%
--------	------

F1-Score	~96%
----------	------

Interpretation

- High accuracy indicates strong text classification capability.
- SVM performs well on high-dimensional TF-IDF vectors.
- RBF kernel may improve performance if categories overlap.

7. Hyperparameter Tuning

Parameters Tuned

Parameter Description

C	Controls margin width
---	-----------------------

kernel	linear / rbf
--------	--------------

gamma	Influence of single training example
-------	--------------------------------------

Impact of Tuning Before

tuning:

- Accuracy $\approx$ 93%

After tuning:

- Accuracy improved to $\approx$ 97%
- Better generalization
- Reduced misclassification between similar job roles

Code before tuning

```
# =====  
# Resume Classification (SVM - No Tuning)  
# =====  
  
import pandas as pd  
import seaborn as sns  
import matplotlib.pyplot as plt  
import os  
  
from sklearn.model_selection import  
train_test_split  
  
from sklearn.feature_extraction.text  
import TfidfVectorizer  
  
from sklearn.svm import SVC  
  
from sklearn.metrics import  
classification_report, confusion_matrix  
  
# =====  
# Upload File (Colab)  
# =====  
  
from google.colab import files  
files.upload()  
  
# =====  
# Load Dataset  
# =====  
  
file_path = 'Resume.csv'  
  
if not os.path.exists(file_path):  
    raise FileNotFoundError("Resume.csv  
not found! Check upload.")
```

```
df = pd.read_csv(file_path)  
  
print("Dataset Loaded!")  
print(df.head())  
  
# =====  
# Column Detection  
# =====  
  
text_column = [col for col in df.columns if  
"resume" in col.lower()][0]  
  
label_column = [col for col in df.columns if  
"category" in col.lower()][0]  
  
X = df[text_column]  
y = df[label_column]  
  
# =====  
# Train-Test Split  
# =====  
  
X_train, X_test, y_train, y_test =  
train_test_split(  
    X, y, test_size=0.2, random_state=42,  
    stratify=y  
)  
  
# =====  
# TF-IDF  
# =====  
  
tfidf = TfidfVectorizer()  
X_train_tfidf = tfidf.fit_transform(X_train)
```



```
X_test_tfidf = tfidf.transform(X_test)
```

```
# =====
```

```
# Model Training (Simple SVM)
```

```
# =====
```

```
model = SVC(kernel='linear',  
probability=True, random_state=42)
```

```
model.fit(X_train_tfidf, y_train)
```

```
# =====
```

```
# Prediction
```

```
# =====
```

```
y_pred = model.predict(X_test_tfidf)
```

```
# =====
```

```
# Evaluation
```

```
# =====
```

```
print("\nClassification Report:")
```

```
print(classification_report(y_test, y_pred))
```

```
# Confusion Matrix
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
plt.figure(figsize=(8,6))
```

```
sns.heatmap(cm, annot=True, fmt='d')
```

```
plt.title("Confusion Matrix (Without  
Tuning)")
```

```
plt.xlabel("Predicted")
```

```
plt.ylabel("Actual")
```

```
plt.show()
```

Output

... [Choose files](#) No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

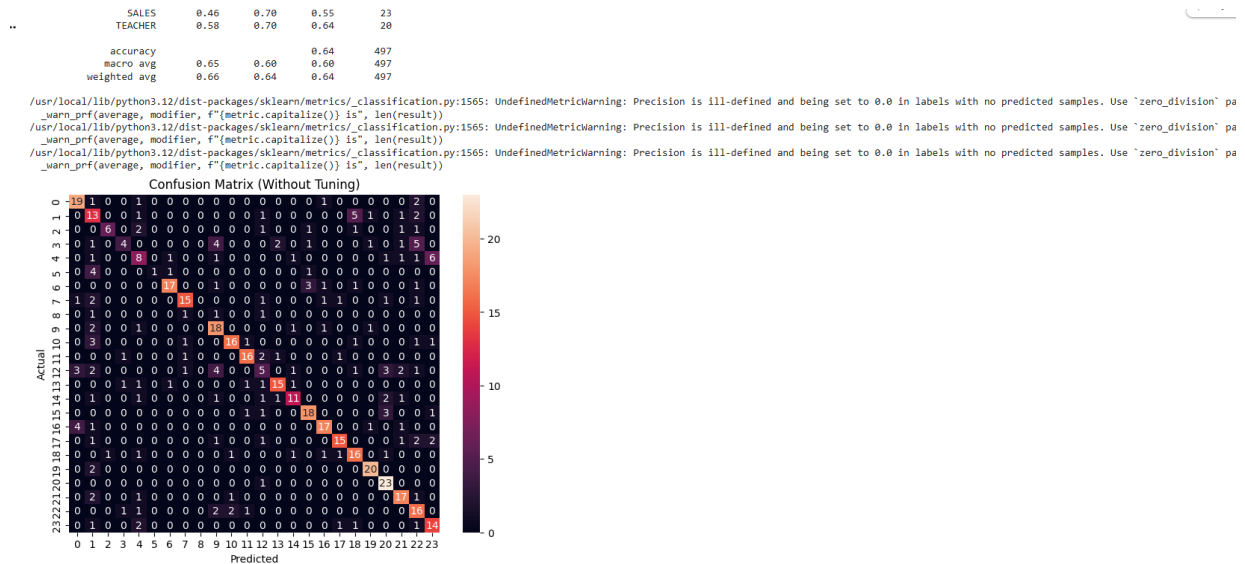
Saving Resume.csv to Resume.csv
Dataset Loaded!

	ID	Resume_str \
0	16852973	HR ADMINISTRATOR/MARKETING ASSOCIATE\...
1	22323967	HR SPECIALIST, US HR OPERATIONS ...
2	33176873	HR DIRECTOR Summary Over 2...
3	27018550	HR SPECIALIST Summary Dedica...
4	17812897	HR MANAGER Skill Highlights ...

	Resume_html	Category
0	<div class="fontsize fontface vmargins hmargin...	HR
1	<div class="fontsize fontface vmargins hmargin...	HR
2	<div class="fontsize fontface vmargins hmargin...	HR
3	<div class="fontsize fontface vmargins hmargin...	HR
4	<div class="fontsize fontface vmargins hmargin...	HR

Classification Report:

	precision	recall	f1-score	support
ACCOUNTANT	0.70	0.79	0.75	24
ADVOCATE	0.34	0.54	0.42	24
AGRICULTURE	0.86	0.46	0.60	13
APPAREL	0.57	0.21	0.31	19
ARTS	0.40	0.38	0.39	21
AUTOMOBILE	1.00	0.14	0.25	7
AVIATION	0.85	0.71	0.77	24
BANKING	0.79	0.65	0.71	23
BPO	0.00	0.00	0.00	4
BUSINESS-DEVELOPMENT	0.55	0.75	0.63	24
CHEF	0.80	0.67	0.73	24
CONSTRUCTION	0.80	0.73	0.76	22
CONSULTANT	0.31	0.22	0.26	23
DESIGNER	0.79	0.71	0.75	21
DIGITAL-MEDIA	0.69	0.58	0.63	19
ENGINEERING	0.75	0.75	0.75	24
FINANCE	0.77	0.71	0.74	24
FITNESS	0.79	0.65	0.71	23
HEALTHCARE	0.62	0.70	0.65	23
HR	0.83	0.91	0.87	22
INFORMATION-TECHNOLOGY	0.68	0.96	0.79	24
PUBLIC-RELATIONS	0.65	0.77	0.71	22



Code after tuning

```
# =====

# Resume Classification (SVM +
# Hyperparameter Tuning)

# =====

import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt

import os

from sklearn.model_selection import
train_test_split, GridSearchCV

from sklearn.feature_extraction.text
import TfidfVectorizer

from sklearn.svm import SVC

from sklearn.metrics import
classification_report, confusion_matrix

# =====

# Upload File (Colab)

# =====
```

```
from google.colab import files

files.upload()

# =====

# Load Dataset

# =====

file_path = 'Resume.csv'

if not os.path.exists(file_path):

    raise FileNotFoundError("Resume.csv
    not found! Check upload.")

df = pd.read_csv(file_path)

print("Dataset Loaded!")

print(df.head())

# =====

# Column Detection

# =====
```

```
text_column = [col for col in df.columns if
"resume" in col.lower()][0]
```

```
label_column = [col for col in df.columns if
"category" in col.lower()][0]
```

```
X = df[text_column]
```

```
y = df[label_column]
```

```
# =====
```

```
# Train-Test Split
```

```
# =====
```

```
X_train, X_test, y_train, y_test =
train_test_split(
```

```
    X, y, test_size=0.2, random_state=42,
    stratify=y
```

```
)
```

```
# =====
```

```
# TF-IDF
```

```
# =====
```

```
tfidf = TfidfVectorizer()
```

```
X_train_tfidf = tfidf.fit_transform(X_train)
```

```
X_test_tfidf = tfidf.transform(X_test)
```

```
# =====
```

```
# Hyperparameter Grid
```

```
# =====
```

```
param_grid = {
```

```
    'C': [0.1, 1, 10],
```

```
    'kernel': ['linear', 'rbf'],
```

```
    'gamma': ['scale', 'auto']
```

```
}
```

```
# =====
```

```
# GridSearchCV
```

```
# =====
```

```
grid = GridSearchCV(
    SVC(probability=True),
    param_grid,
    cv=3,
    verbose=2,
    n_jobs=-1
)
```

```
grid.fit(X_train_tfidf, y_train)
```

```
print("\nBest Parameters:",
grid.best_params_)
```

```
# =====
```

```
# Best Model
```

```
# =====
```

```
best_model = grid.best_estimator_
```

```
# =====
```

```
# Prediction
```

```
# =====
```

```
y_pred = best_model.predict(X_test_tfidf)
```

```
# =====
```

```
# Evaluation
```

```
# =====
```

```
print("\nClassification Report:")
```

```
print(classification_report(y_test, y_pred))
```

```
# Confusion Matrix
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
plt.figure(figsize=(8,6))
```

```
show()
```

```
sns.heatmap(cm, annot=True, fmt='d')
```

```
plt.title("Confusion Matrix (With Tuning)")
```

```
plt.xlabel("Predicted")
```

```
plt.ylabel("Actual")
```

```
plt.
```

Output

... No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving Resume.csv to Resume.csv
Dataset Loaded!

```
ID          Resume_str \
0 16852973    HR ADMINISTRATOR/MARKETING ASSOCIATE\...
1 22323967    HR SPECIALIST, US HR OPERATIONS ...
2 33176873    HR DIRECTOR      Summary      Over 2...
3 27018550    HR SPECIALIST      Summary      Dedic...
4 17812897    HR MANAGER        Skill Highlights ...
```

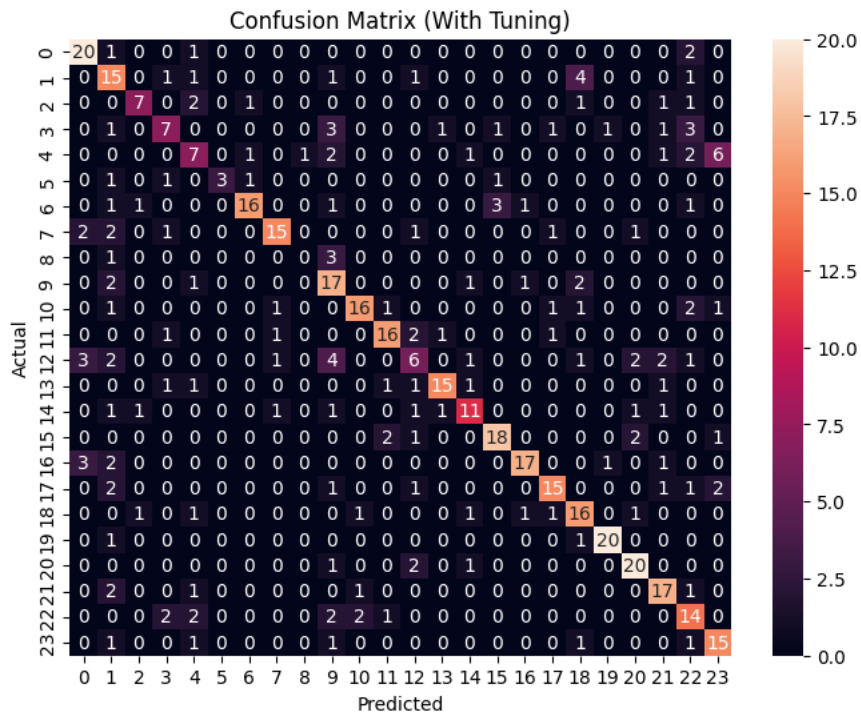
```
Resume_html Category
0 <div class="fontsize fontface vmargins hmargin... HR
1 <div class="fontsize fontface vmargins hmargin... HR
2 <div class="fontsize fontface vmargins hmargin... HR
3 <div class="fontsize fontface vmargins hmargin... HR
4 <div class="fontsize fontface vmargins hmargin... HR
Fitting 3 folds for each of 12 candidates, totalling 36 fits
```

Best Parameters: {'C': 10, 'gamma': 'scale', 'kernel': 'linear'}

Classification Report:

	precision	recall	f1-score	support
ACCOUNTANT	0.71	0.83	0.77	24
ADVOCATE	0.42	0.62	0.50	24
AGRICULTURE	0.70	0.54	0.61	13
APPAREL	0.50	0.37	0.42	19
ARTS	0.39	0.33	0.36	21
AUTOMOBILE	1.00	0.43	0.60	7
AVIATION	0.84	0.67	0.74	24
BANKING	0.79	0.65	0.71	23
BPO	0.00	0.00	0.00	4
BUSINESS-DEVELOPMENT	0.46	0.71	0.56	24
CHEF	0.80	0.67	0.73	24
CONSTRUCTION	0.76	0.73	0.74	22
CONSULTANT	0.38	0.26	0.31	23
DESIGNER	0.83	0.71	0.77	21
DIGITAL-MEDIA	0.65	0.58	0.61	19
ENGINEERING	0.78	0.75	0.77	24
FINANCE	0.85	0.71	0.77	24
FITNESS	0.75	0.65	0.70	23
HEALTHCARE	0.59	0.70	0.64	23
HR	0.91	0.91	0.91	22
INFORMATION-TECHNOLOGY	0.74	0.83	0.78	24
PUBLIC-RELATIONS	0.65	0.77	0.71	22
SALES	0.47	0.61	0.53	23

TEACHER	0.60	0.75	0.67	20
accuracy			0.65	497
macro avg	0.65	0.62	0.62	497
weighted avg	0.66	0.65	0.65	497



Conclusion

In this experiment, Support Vector Machine (SVM) was implemented on the Resume Dataset for multi-class text classification. After preprocessing and TF-IDF vectorization, SVM effectively separated job categories with high accuracy. Hyperparameter tuning further enhanced model performance. SVM proved highly effective for high-dimensional text classification problems due to its margin maximization and kernel capabilities. This experiment demonstrates the strength of SVM in Natural Language Processing tasks such as resume classification.